

The Power of Playwright

Scaling UI Testing for Robust Application Delivery



Let's Connect!



Randy-Pagels



PagelsR



<https://xebia.com>



@RandyPagels



Randy Pagels

DevOps Architect | Trainer
Randy.Pagels@Xebia.com

Agenda

- Testing Challenges & Goals
- Playwright Overview
- Getting Started
- Authoring, Debugging, Running
- Additional Built in Features
- Time Travel Experience
- Continuous Integration Testing
- Playwright Testing Service



UI Testing Challenges

Consistent design across browsers!
Dynamic content and maintaining performance!

- Testing is hard
- Testing takes time to learn
- Testing takes time to build
- Tests are slow – they take too long to run!
- Tests are flaky – they crash all the time!
- Tests are brittle – they break whenever the app changes!



Modern Testing Goals

Major goals for modern web testing

- Make testing easy!
- Make testing fun!
- Focus on fast feedback loops!
- Make test development as painless as possible!
- Testing tool that complements dev workflows!
- Testing tool that easily integrates with CI/CD!
- Testing that supports Testing At Scale!



Test Automation Practices

Tests should be automated when they give positive ROI

Advantages

- Tests can be **rerun at any time** the same way.
- Tests can run as **part of CI/CD**.
- **Setup** and **cleanup** can be **automated**.
- Frameworks can provide reports, logs, videos, and screenshots.

ProTips!

- Each **test** should **focus** on **one main behavior** or variation.
- **Tests** should **run independently** of each other and in any order.
- **Tests** should be **self-descriptive** and intuitively understandable.
- Do not automate every test – use a risk-based strategy with ROI.
- **Tests** should **run** in **parallel** at **scale**, especially in CI/CD.



AN INTRODUCTION TO PLAYWRIGHT

@playwrightweb
<https://playwright.dev>



5
RESOURCES
TO HELP YOU
JUMPSTART
YOUR TEST
ADVENTURE

1- READ THE DOCS
• <https://aka.ms/playwright>
• <https://aka.ms/playwright/quickstart>
• Install Playwright
• Run your first test
• Explore Tools & Libraries

2- EXPLORE THE API
<https://aka.ms/playwright/api>
• Playwright Test Runner
• Test Reporter
• Testing + Experimental

3- VISIT THE REPO
<https://aka.ms/playwright/github>
• Playwright Framework source
• Examples + Release docs
• Discussions

4- FOLLOW @playwrightweb
• Framework release updates
• Content + event announcements
• Community interactions

5- CHECKOUT DEMO CODE
<https://aka.ms/playwright/demo>
• Test script examples
• GitHub Actions integration
• Test Reporting outputs

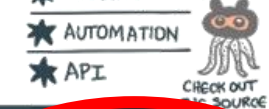
WHAT IS
PLAYWRIGHT?

- 1 AN OPEN SOURCE TESTING FRAMEWORK
- 2 ENABLES RELIABLE END-TO-END TESTING AND AUTOMATION
- 3 FOR MODERN WEB APPLICATIONS

<https://github.com/microsoft/playwright>

- ★ TOOLING
- ★ AUTOMATION
- ★ API

CHECK OUT THE SOURCE



5
POWERFUL
TOOLING!

1 CODE GEN
• GENERATE TESTS BY RECORDING USER ACTIONS (FLAKY)
• SAVE IN ANY SUPPORTED LANG. (JAVASCRIPT, PYTHON, C#)

2 INSPECTOR
• INSPECT PAGE, LOGS
• GENERATE SELECTOR
• STEP THROUGH EXEC
• SEE CLICK POINTS

3 TRACE VIEWER
• CAPTURE TEST PROGRESS (STATE, FAILURES, TIMING)
• SCREENCASTS, SNAPSHOTS, ACTION EXPLORER, TEST SOURCE (POST-MORTEM ANALYSIS)

4
FULL ISOLATION
FAST EXECUTION

1 BROWSER CONTEXT
• CREATES A BROWSER CONTEXT PER TEST IN JUST MS.
• FULL TEST ISOLATION
• ZERO OVERHEADS
• LESS COMPLEXITY FOR TESTER

2 LOG IN ONCE!
• SAVE AUTHENTICATION STATE (tokens, cookies) FROM FIRST RUN → REUSE IN ALL SUBSEQUENT TESTS
• BYPASS REDUNDANT LOGIN OPERATIONS (many third-party sites don't want automated logins)
• STILL FULL ISOLATION FOR INDEPENDENT TESTS

WHY SHOULD I
BE USING
PLAYWRIGHT?

- 1 ONE API, MULTI-EVERYTHING
- 2 RESILIENT, NO FLAKY TESTS
- 3 LIMITLESS, NO TRADEOFFS
- 4 ISOLATED, FAST EXECUTION
- 5 TOOLING, CREATE-DEBUG-ANALYZE

3
NO TRADE-OFFS
LIMITS

1 TESTED WITH MODERN WEB ARCHITECTURES
• RUN TESTS OUT OF PROCESS - FREED FROM IN-PROCESS TEST RUNNER CONSTRAINT

2 MULTIPLE EVERYTHING
• MULTIPLE TABS
• MULTIPLE USERS
• TEST ALL THE THINGS

3 TRUSTED EVENTS
• TEST HOOK ELEMENTS
• INTERACT WITH DYNAMIC CONTROLS
• USE REAL BROWSER INPUT PIPELINES (INDISTINGUISHABLE FROM REAL USERS!)

4 TEST FRAMES
• PIECE SHADOW DOMS WITH PLAYWRIGHT SELECTORS!

1
ONE API
MULTI-
EVERYTHING

ANY BROWSER
ANY PLATFORM
Chromium, WebKit, Firefox
MacOS, Linux, Windows

2
RESILIENT!
NO FLAKY TESTS...

1 AUTO-WAIT
No more artificial timeouts to manage
waits for elements to be ACTIONABLE before execution of test steps

2 WEB-FIRST ASSERTIONS
tailored for the dynamic web!
+ RICH introspection events available!
convenience async matchers for assertions
separate timeout (from test) for web first assert with default = 5 secs
checks conditions till met

3 TRACING
configurable strategy
record, capture, process

ANY BROWSER
ANY PLATFORM
Chromium, WebKit, Firefox
MacOS, Linux, Windows

CROSS LANGUAGE
JavaScript, TypeScript, Python, Java, .NET - Libraries

TEST MOBILE WEB
Native emulation
Chrome on Android
Mobile Safari on iOS
Same Rendering Engine on Cloud and Desktop

CROSS BROWSER TESTING
TEST AUTOMATION AT CLOUD SCALE

TEST ACTIONS
ARE YOU EXPECTING US?
- LET ME CHECK AGAIN...

REDUCE TEST COMPLEXITY
INCREASE RELIABILITY

TESTING ENTRY
ALMOST READY FOR YOU - HANG ON - LET ME CHECK AGAIN...

WEB PAGE

ILLUSTRATED BY @SKETCHTHEDOCS

Playwright Overview

Open-source web test automation library on Node.js, which makes test automation easier for browsers based on Chromium, Firefox, and Webkit through a single API.

- ✓ **Cross-Browser**
 - Chromium, Firefox, and WebKit
- ✓ **Cross-language**
 - JavaScript, Python, Java, and C#
- ✓ **Cross-platform**
 - Windows, Linux, and macOS
- ✓ **Modern **Asynchronous API****
 - `async/await`
- ✓ **Rich Element Interaction**
 - wide range of built-in functions
- ✓ **Screenshots and Video Recording**
 - visual evidence of issues
- ✓ **Full Isolation – Fast Execution**
 - separate browser contexts for each test



Reliable end-to-end testing for modern web apps

...by Microsoft

...and the people who made Puppeteer

<https://playwright.dev/>

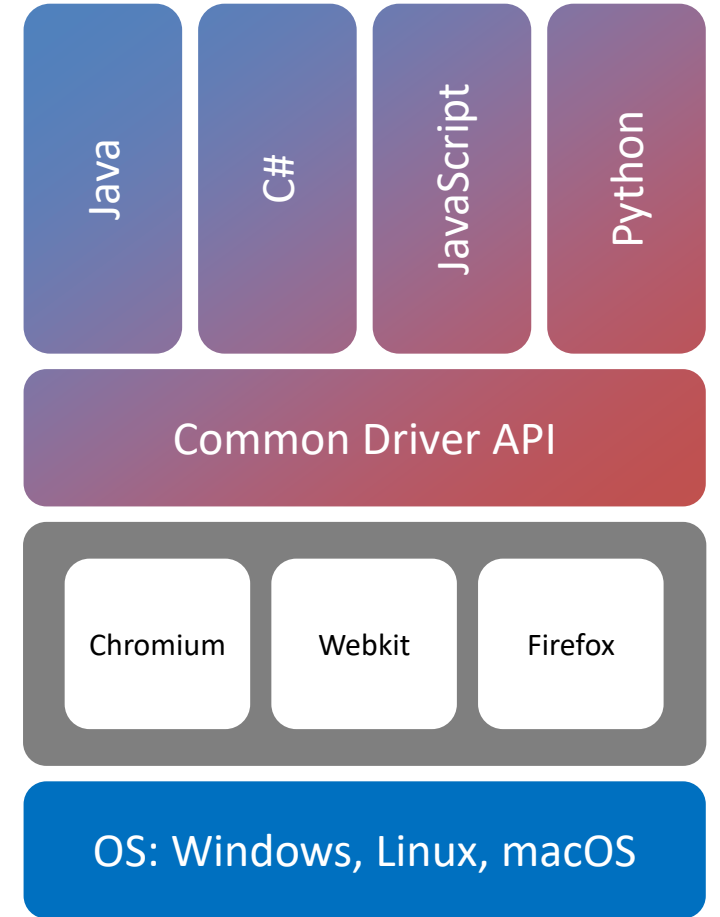
<https://github.com/microsoft/playwright>

Playwright Architecture

✓ Language Bindings →

✓ Single Automation Protocol ✓ Unify all debugging protocols →

✓ (Native) browser debugging e.g. ChromeDevTools Protocol, protocols, (Safari) Web Inspector →



Getting Started

Setting up using VS Code

Install Extension

Install Playwright

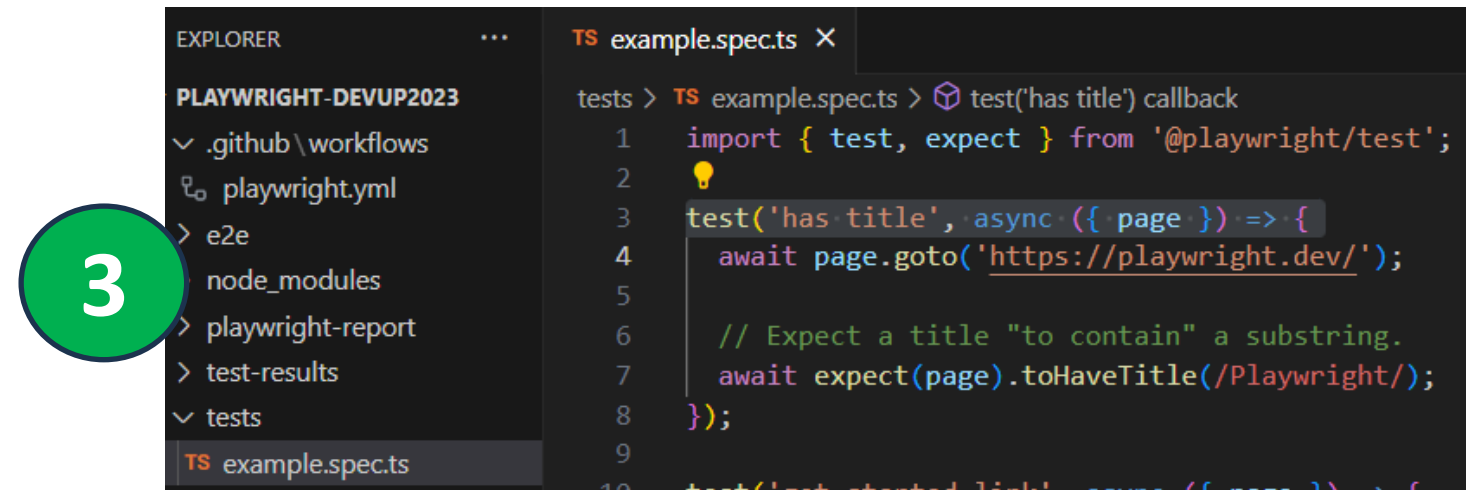
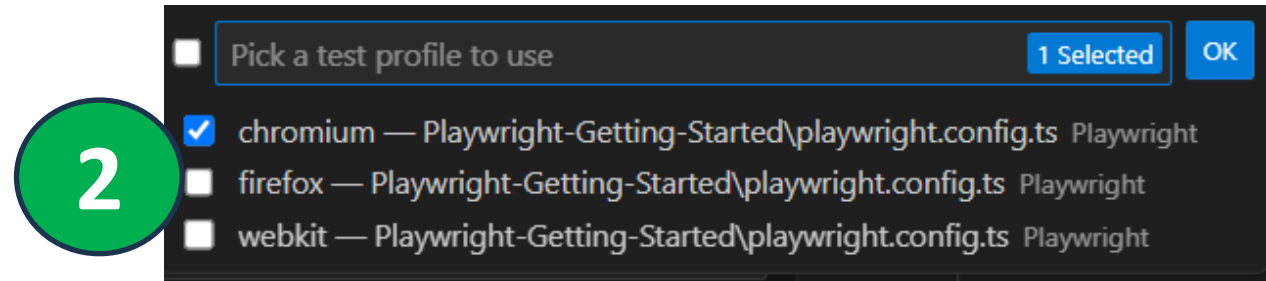
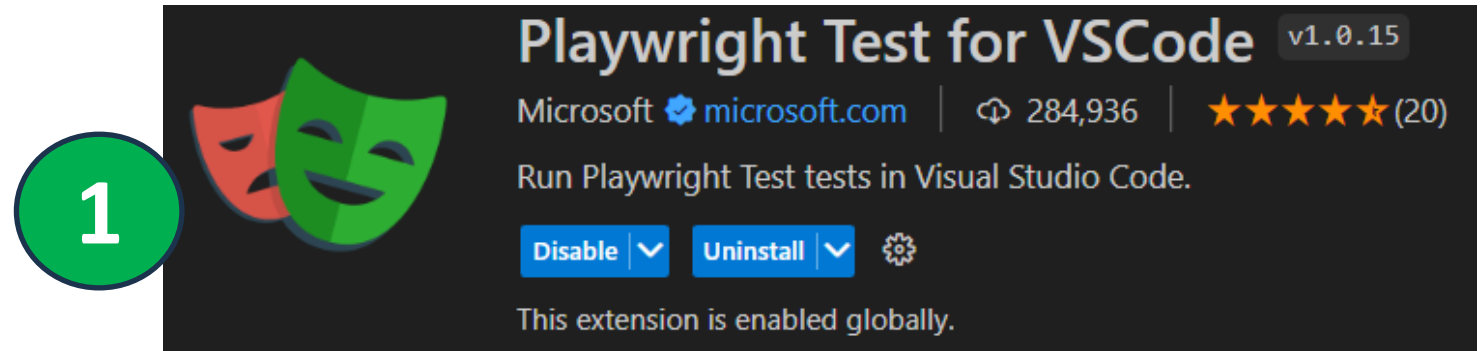
- `npm init playwright@latest`

Run Tests

- `npx playwright test`
- `npx playwright test --ui`

Test Reports

- `npx playwright show-report`



Demo

- ✓ Getting Started
- ✓ Generate Tests
- ✓ Debugging
- ✓ Trace Viewer
- ✓ Test Report
- ✓ UI Mode – Visual Feedback



Automating Manual Testing



Azure DevOps Test Plans with Playwright

Publish your automated Playwright test results to Azure Test Plans

- ✓ Associate automated tests with Test Cases!
- ✓ Using Azure Test Plans with Playwright!
- **End-to-End Traceability**: Requirements (PBI's, User Stories) in Azure Boards are linked to test cases.
- **User-Friendly Testing**: Pipelines enabling the triggering of automated tests through Test Cases, rather than pipelines.
- **History Access**: Azure Test Plans offers access to historical test data, simplifying progress tracking through reports and charts.
- **Test Inventory Management**: Both automated and manual tests are managed efficiently, enabling effective tracking of their status and the progress of automation efforts.

Test Case Setup

Publish your automated Playwright tests to the ADO service

Manually create new test cases in Azure Test Plans

1. Create manual Test Cases
2. Add the **Test Case ID** in brackets to the test case title
3. Publish test results from build pipeline

The screenshot shows the Azure Test Plans interface. On the left, a sidebar lists test suites under 'eShopOnWeb2024_TestPlan1 (1)'. The main area displays a test suite titled '11159 : Shop by Brand on the eShopOnWeb portal (ID: 11161)'. Below the title, there are tabs for 'Define', 'Execute', and 'Chart'. A table titled 'Test Points (6 items)' is shown, with columns: Title, Outcome, Order, and Test Case Id. The table contains six rows of test points. A green circle with the number 1 is overlaid on the table.

Title	Outcome	Order	Test Case Id
☒ Shop by Brand - All	Active	1	11185
☒ Shop by Brand - .Net	Active	2	11186
☐ Shop by Brand - Azure	Active	3	11187
☐ Shop by Brand - Other	Active	4	11188
☐ Shop by Brand - SQL Server	Active	5	11189
☐ Shop by Brand - Visual Studio	Active	6	11190

The screenshot shows Playwright test code. Two red boxes highlight the test case IDs in the test titles. A green circle with the number 2 is overlaid on the first box, and a green circle with the number 3 is overlaid on the second box.

```
7
8 test.describe('Header', () => {
9
10 > test('[11185] Shop by Brand - All', async ({ page }) => {
11     // ...
12 });
13
14 test('[11186] Shop by Brand - .Net', async ({ page }) => {
15     // ...
16     await expect(page).toHaveTitle('Catalog - Microsoft');
17 });
18 }
```

```
1 Starting: publish test results
2 =====
3 Task      : Publish Test Results
4 Description : Publish test results to Azure Pipelines
5 Version    : 2.233.1
6 Author     : Microsoft Corporation
7 Help       : https://docs.microsoft.com/azure/devops/pipelines/build/tasks/publish-test-results
8 =====
9 Result Attachments will be stored in LogStore
10 Run Attachments will be stored in LogStore
11 Async Command Start: Publish test results
12 Publishing test results to test run '1016595'.
13 TestResults To Publish 47, Test run id:1016595
14 Test results publishing 47, remaining: 0. Test run
15 Published Test Run : https://dev.azure.com/xpirit/eShopOnWeb2024/_build/results?buildId=1016595
16 Async Command End: Publish test results
```


Text Execution History

11149 : Login and new user registraion to the eShopOnWeb portal using email id and password (ID: 11156)

Define

Execute

Chart

Test Points (8 items)

Title

User Registration

User Account Login for General User

Invalid Login Attempt

User Account Login for Admin User

Run for web app

Passed

Failed

Failed

Passed

View execution history

Mark Outcome

Run

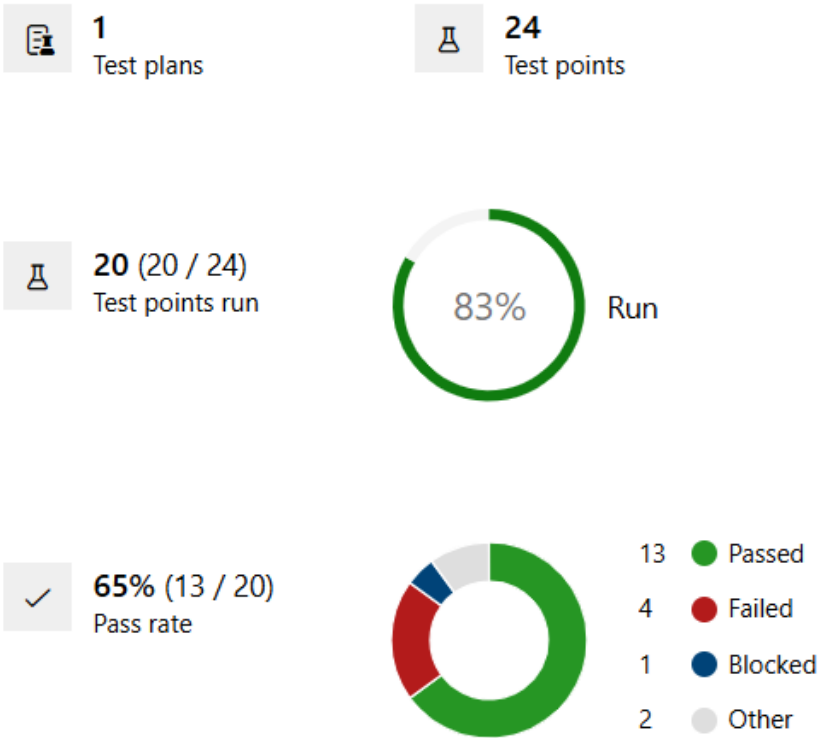
3

Test Case Results

Outcome	TimeStamp	Configuration	Run by
Failed	1h ago	Windows 10	Randy Pagels
Failed	1h ago	Windows 11	Randy Pagels
Passed	1h ago	Windows 11	Randy Pagels
Passed	1h ago	Windows 10	Randy Pagels
Passed	1h ago	Windows 10	Randy Pagels
Passed	1h ago	Windows 11	Randy Pagels
Passed	4h ago	Windows 11	Randy Pagels
Passed	4h ago	Windows 10	Randy Pagels
Passed	4h ago	Windows 10	Randy Pagels
Passed	4h ago	Windows 11	Randy Pagels
Passed	7h ago	Windows 11	Randy Pagels
Passed	7h ago	Windows 10	Randy Pagels
Passed	7h ago	Windows 11	Randy Pagels

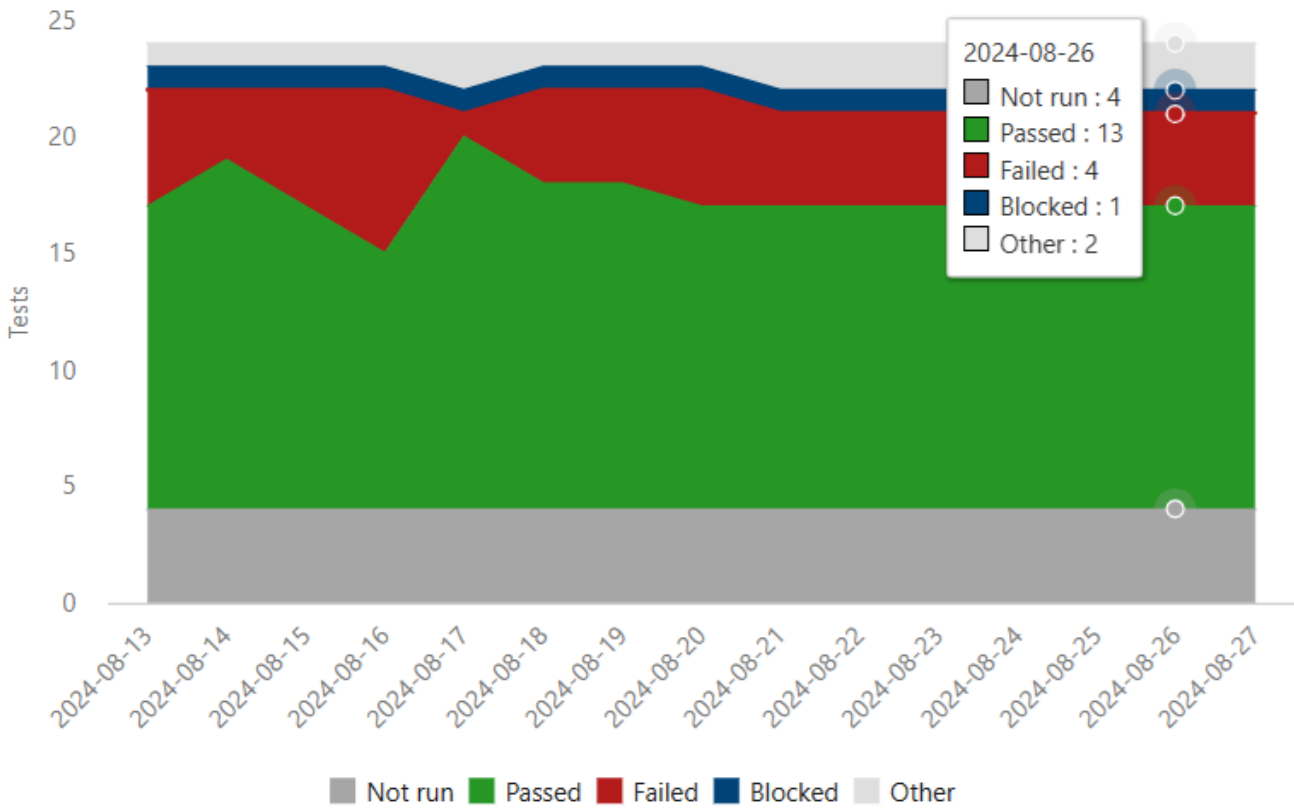
Test Plans Progress Report

Summary



Outcome trend

Last 14 Days



Demo

- ✓ Automate Test Cases using Playwright
- ✓ Test Execution History
- ✓ Progress Report



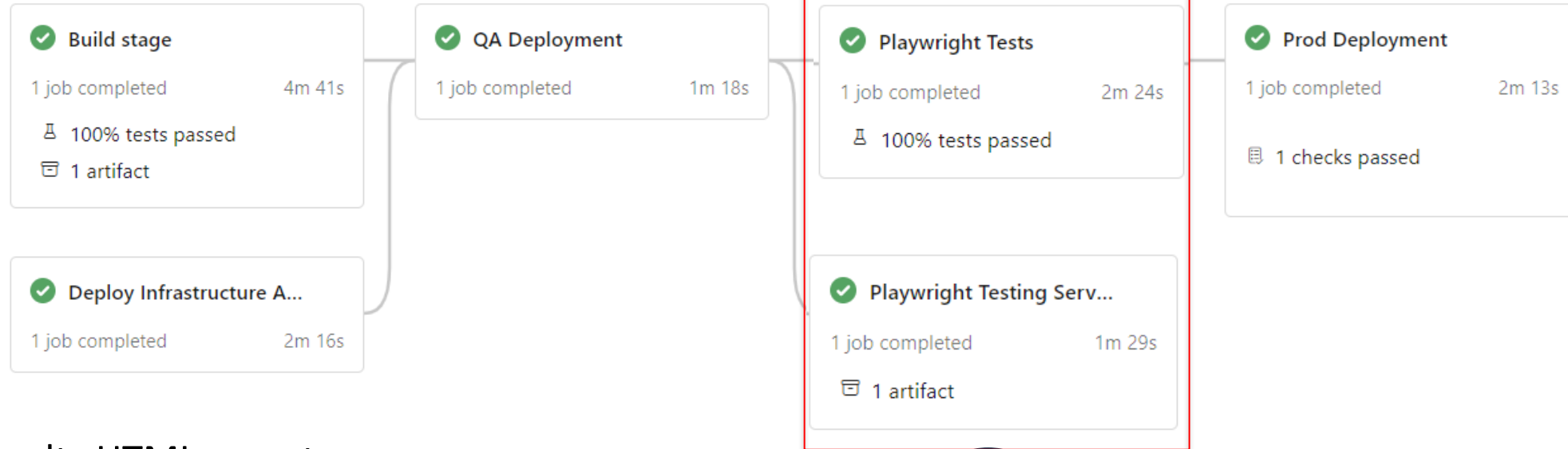
Continuous Integration Testing at Scale



Continuous Testing

Tests run on each push and pull request into the main/main branch

- ✓ Installs all dependencies
- ✓ Installs Playwright and Browsers
- ✓ Parallelization for Test Suites
- ✓ Run all the tests

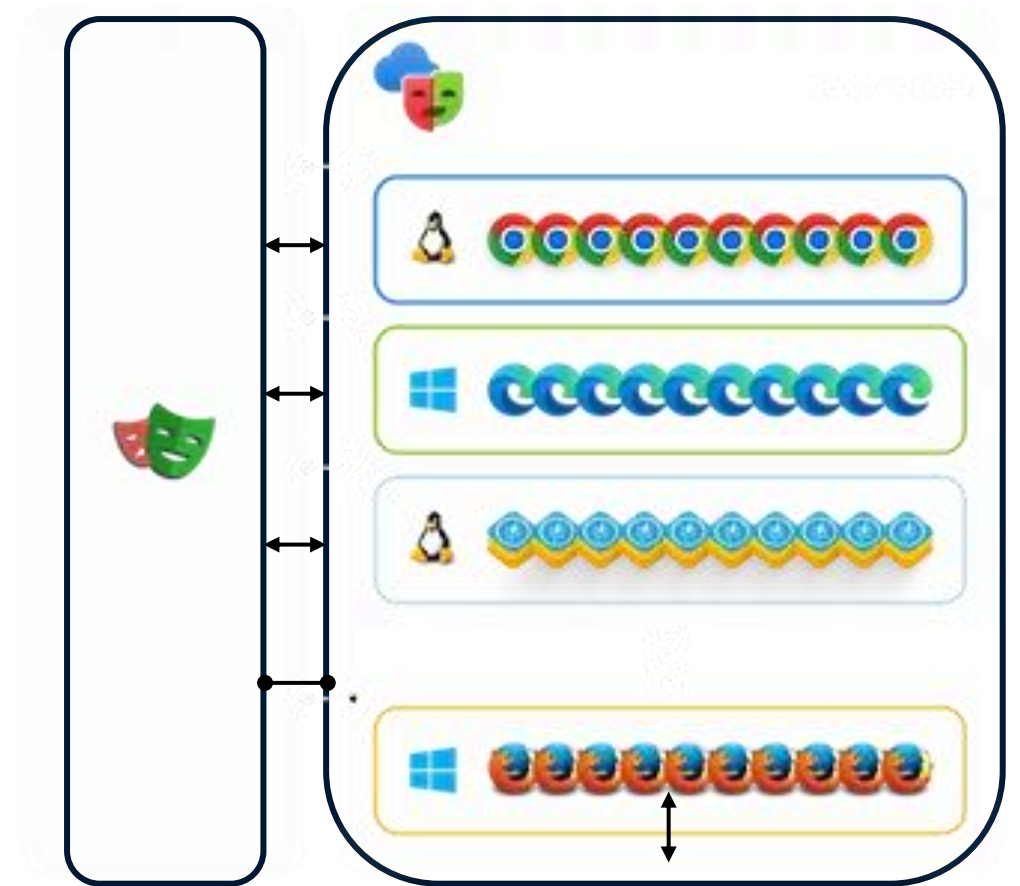


- ✓ Generate Test Results HTML report
- ✓ Generate Screenshots and Video Recordings
- ✓ Generate Debug Trace file
- ✓ Deploy static files to Website or GitHub Pages for viewing

Microsoft Playwright Testing

Scalable end-to-end modern web app testing service

- Run Playwright tests flexibly in the cloud
- Run Playwright tests with browser and OS combinations
- Cloud enabled to run Playwright tests at scale.
- High parallelization across operating system-browser combinations.
- Get tests done much faster.
- Speed up delivery of features without sacrificing quality.



Demo

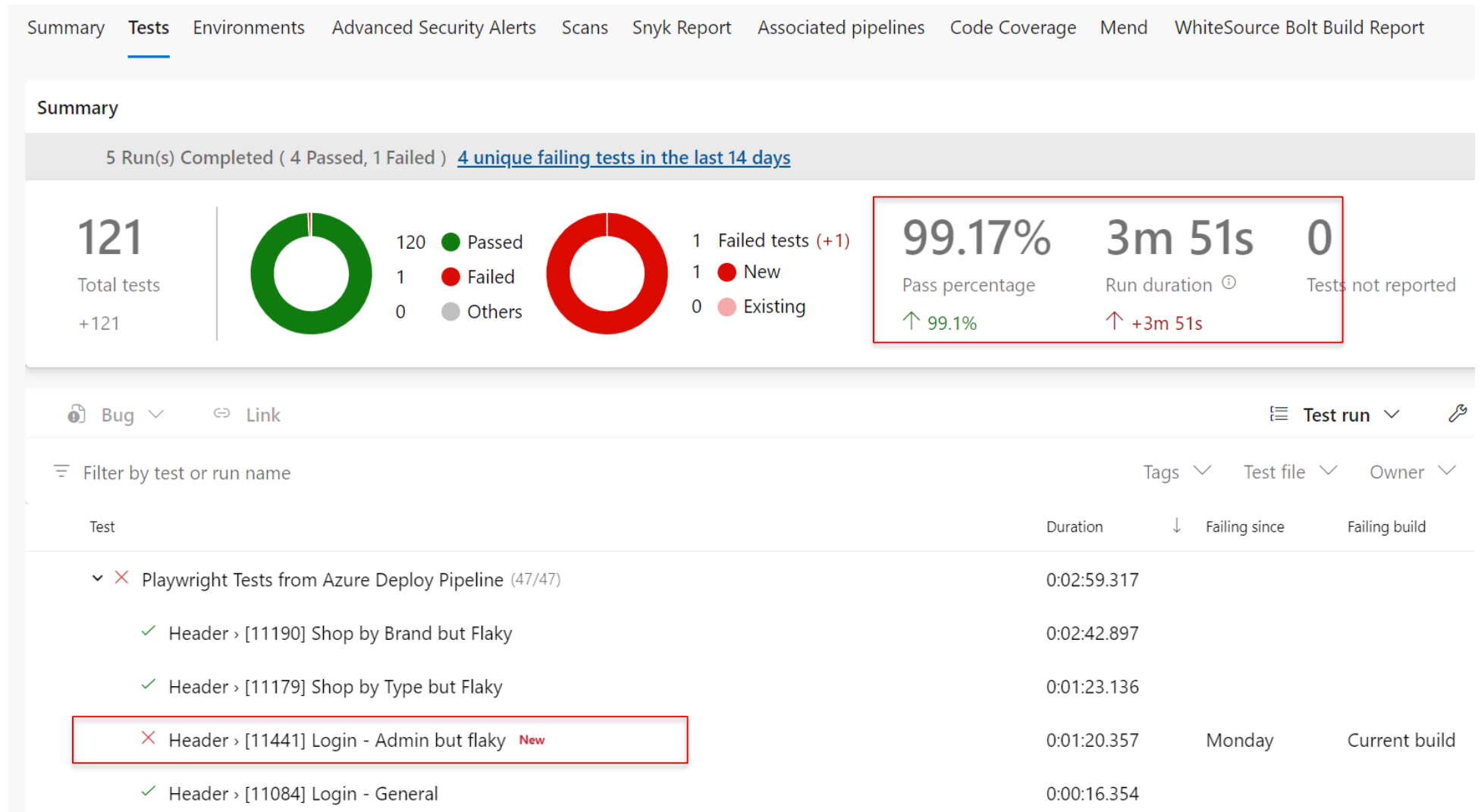
- ✓ Build and Deploy Pipeline
- ✓ Pipeline Results
- ✓ Playwright Testing Service, Testing as Scale
- ✓ Playwright Testing Service Reporting (Private Preview)
- ✓ HTML Report on GitHub Pages for viewing



Pipeline Results

Per build

1. Passed!
2. Failed!
3. Percentage!



Microsoft Playwright Testing Rich Reporting

Test results filtered by failed and flaky tests

Reporting Private Preview

Update Azure Pipelines configuration for Playwright tests

⊗ Test run failed

✓ 45 Passed

⊗ 3 Failed

⚠ 0 Flaky

⊖ 0 Skipped

🕒 1m:17.9s

👤 50 workers

📅 10th August 5:51 ...

Test summary

🔍 S:Failed ✕

S:Passed ✕

All (48)

✓ Passed (45)

⊗ Failed (3)

⚠ Flaky (0)

⊖ Skipped (0)

☰ Project

Test case	Project	Duration
▼ ShopByBrand.spec.ts (7)		
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByBra... > [11188] Shop by Brand - ...	chromium	🕒 0m:22.9s
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByBra... > [11186] Shop by Brand ...	chromium	🕒 0m:19.6s
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByBra... > [11187] Shop by Brand - ...	chromium	🕒 0m:15.6s
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByB... > [11189] Shop by Brand - S...	chromium	🕒 0m:23.1s
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByBra... > [11185] Shop by Bran...	chromium	🕒 0m:17.9s
⊗ ShopByBrand.spec... - Header>PlaywrightTests\ShopByBra... > [11190] Shop by Brand b...	chromium	🕒 1m:00s
✓ ShopByBrand.spec... - Header>PlaywrightTests\ShopByB... > [11190] Shop by Brand - Vis...	chromium	🕒 0m:18.8s

Microsoft Playwright Testing Rich Reporting

A test result with CI information

Reporting Private Preview

The screenshot displays the Microsoft Playwright Testing Rich Reporting interface. The left sidebar shows a navigation menu with 'Home / Test run / Test case' and a summary of '45 Passed' tests. The main panel is titled 'Test case' and shows a failed test case '[11190] Shop by Brand but Flaky' with a 'TimedOut' status and a duration of '1m:00s'. The browser configuration is 'Chrome' on 'Windows' using 'chromium'. The 'Errors' tab is active, showing a 'Test timeout of 60000ms exceeded.' error. A red box highlights the error details: 'Error: page.selectOption: Test timeout of 60000ms exceeded. Call log: - waiting for locator('#CatalogModel_BrandFilterApplied') - locator resolved to <select class="esh-catalog-filter" id="CatalogModel_Bran...></select> - selecting specified option(s) - did not find some options - waiting...'. The 'Test steps' tab shows a list of steps: 'Before Hooks' (16,731 ms), 'expect.toHaveTitle' (144 ms), 'page.selectOption(#CatalogModel_BrandFilterApplied)' (49,390 ms, highlighted with a red arrow), and 'After Hooks' (10,993 ms).

Microsoft Playwright testing PREVIEW

Home / Test run / Test case

Update Azure Pipelines configuration

Test run failed 45 Passed

Test summary

Q S:Failed X S:Passed X S:Flaky

Test case

ShopByBrand.spec.ts (7)

- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts
- ShopByBrand.spec.ts - Header>PlaywrightTests/ShopByBrand.spec.ts

eShopLogin.spec.ts (4)

Test case

[11190] Shop by Brand but Flaky

Header>PlaywrightTests/ShopByBrand.spec.ts

TimedOut

1m:00s

Chrome

windows

chromium

Update Azure Pipelines configuration

Errors Test steps Attachments

Test timeout of 60000ms exceeded.

Error: page.selectOption: Test timeout of 60000ms exceeded.
Call log:
- waiting for locator('#CatalogModel_BrandFilterApplied')
- locator resolved to <select class="esh-catalog-filter" id="CatalogModel_Bran...></select>
- selecting specified option(s)
- did not find some options - waiting...

Test steps

- Before Hooks 16,731 ms
- expect.toHaveTitle PlaywrightTests/ShopByBrand.spec.ts 144 ms
- page.selectOption(#CatalogModel_BrandFilterApplied) PlaywrightTests/ShopByBrand.spec.ts 49,390 ms
- After Hooks 10,993 ms

Playwright Reporting Private Preview

Microsoft Playwright Testing is a service for running Playwright tests easily at scale. It uses the cloud to enable you to run Playwright tests with much higher parallelization across different operating system-browser combinations. The service is currently in preview.

Check out the Microsoft Playwright Testing service at <https://aka.ms/mpt/about>

Reporting Private Preview

Microsoft is building reporting feature for Microsoft Playwright Testing service to enable you to publish test results of your Playwright tests and view them in the service portal.

Join the waitlist for Microsoft Playwright Testing - **Reporting Private Preview**

Sign up at <https://aka.ms/mpt/reporting-signup>

Azure AD Authentication

- Store the credentials of test accounts using environment variables.

```
test(`example test`, async ({ page }) => {  
  // ...  
  await page.getByLabel('User Name').fill(process.env.USERNAME);  
  await page.getByLabel('Password').fill(process.env.PASSWORD);  
});
```

- Set the values of these variables to use your CI system's secret management or Azure Key Vault.

```
testE2E:  
  name: Run Playwright E2E tests  
  runs-on: ubuntu-latest  
  container: mcr.microsoft.com/playwright:v1.27.0-jammy  
  env:  
    USERNAME: ${secrets.USERNAME}  
    PASSWORD: ${secrets.PASSWORD}
```

- Local Development? Use .env files and add to .gitignore.

```
# .env file  
STAGING=0  
USERNAME=me  
PASSWORD=secret
```

Note: MFA cannot be fully automated and requires manual intervention.

Tool Comparison

Playwright is not the only browser automation tool out there

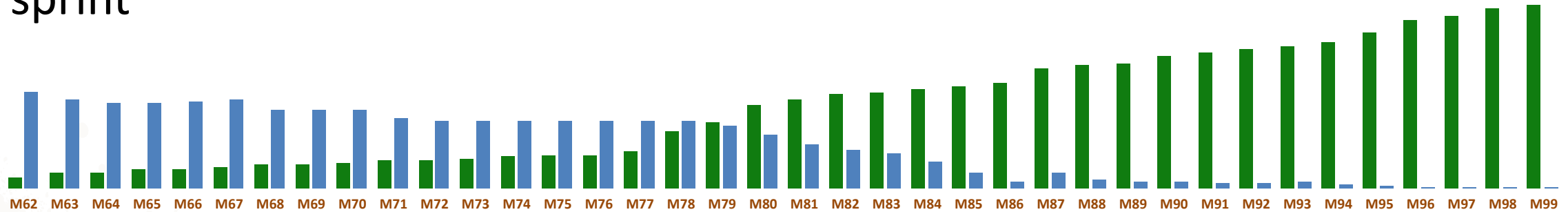
Selenium 	Cypress 	Playwright 
Popular among testers	Popular among developers	So hot right now
C#, Java, JavaScript, Ruby, Python	JavaScript only	C#, Java, JavaScript, Python
All major full browsers	Chrome, Edge, Firefox, Electron	Chromium+, Firefox, WebKit
WebDriver protocol	In-browser JavaScript	Debug protocols
Just a tool – requires a framework	Full, modern framework	Full, modern framework
Can be slow/flaky if waiting is poor	Visual execution but can be slow	Typically the fastest execution
Open source, standards, & gov	Open source from Cypress	Open source from Microsoft

Microsoft Azure DevOps team

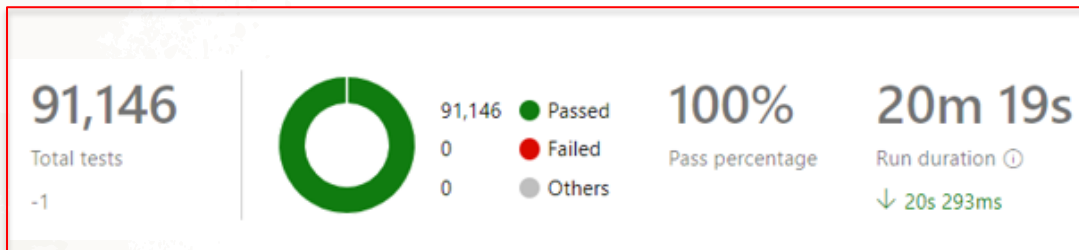
Balancing UI vs Unit Testing in your strategy to maximizes test coverage and effectiveness.

Long running UI
functional tests at end of
sprint

Shifted to unit tests from automated
UI functional tests



ADO PRs require 100% pass rate before merging



ADO Unit Tests in build pipeline for past 14 days



Playwright Resources

Documentation: <https://playwright.dev>

Source / Issues: <https://github.com/microsoft/playwright>

Social Media

- <https://aka.ms/playwright/discord>
- <https://aka.ms/playwright/youtube>
- <https://aka.ms/playwright/x>

Microsoft Playwright Testing

<https://aka.ms/mpt/about>

<https://playwright.microsoft.com/workspaces>

Test Automation with Playwright with Visual Guide Cheatsheet

<https://www.azurestaticwebapps.dev/blog/devtools-playwright>

eShopOnWeb: <https://github.com/dotnet-architecture/eShopOnWeb>

Slides: <https://github.com/PagelsR/Talks> (Mastering Application Testing with Playwright.pdf)

Thank you!

<https://github.com/PagelsR/Talks>

Randy.Pagels@Xebia.com



Appendix

Test Generator

Playwright Test Generator is a GUI tool that records actions performed in the browser and generates code.

CLI

- `npx playwright codegen`

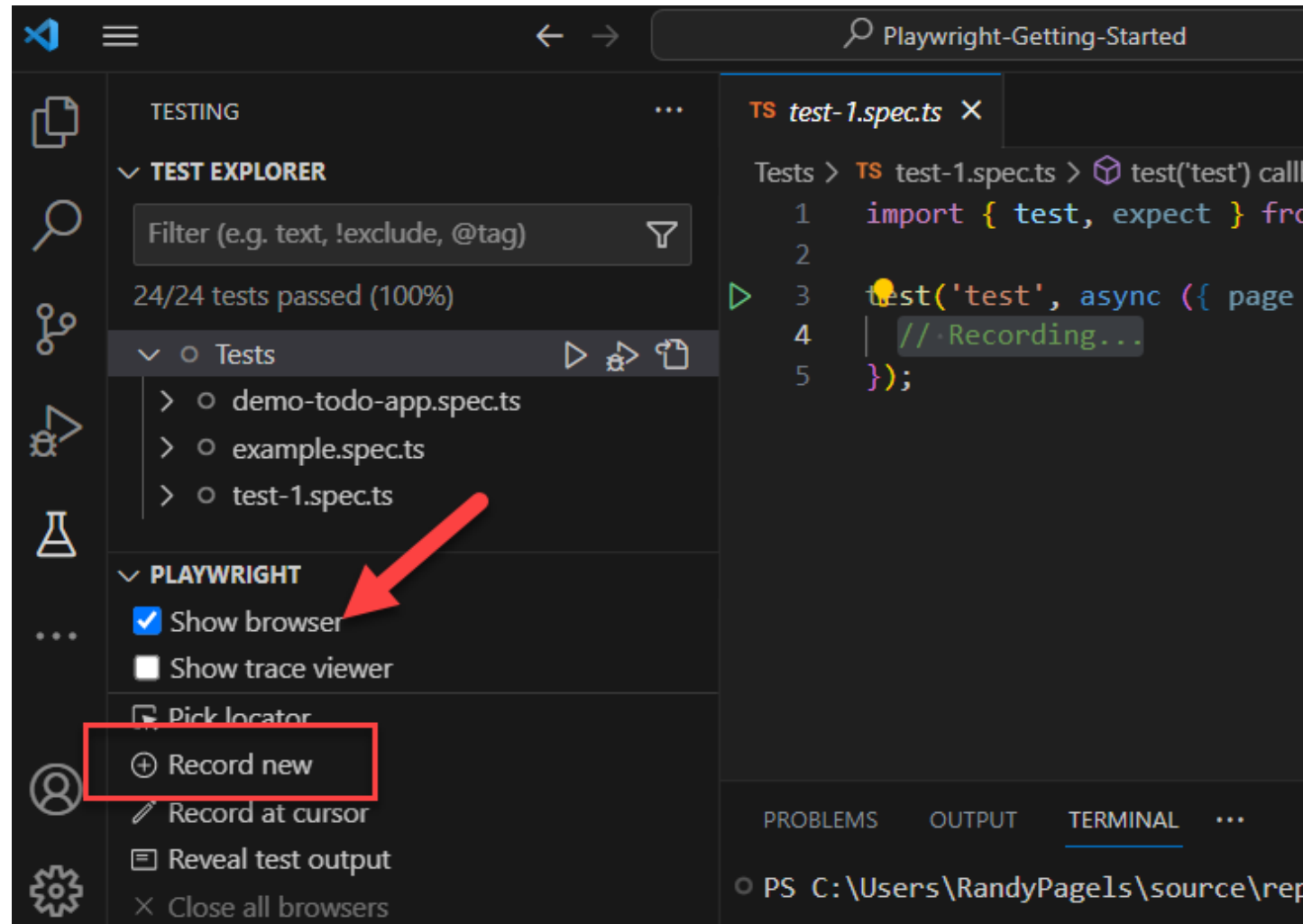
VS Code

- Use Plugin, "Record Now"

With the test generator you can:

- Record Actions
- Assert Elements
- Create Locators
- Emulate Devices

<https://playwright.dev/docs/codegen>



Trace Viewer

Playwright Trace Viewer is a GUI tool that helps you explore recorded Playwright traces after the script has ran.

CLI

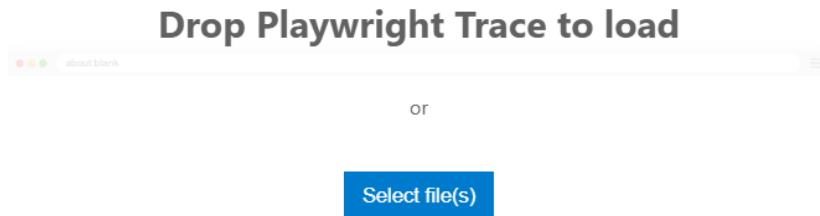
- `npx playwright test test.ts --trace on`

VS Code

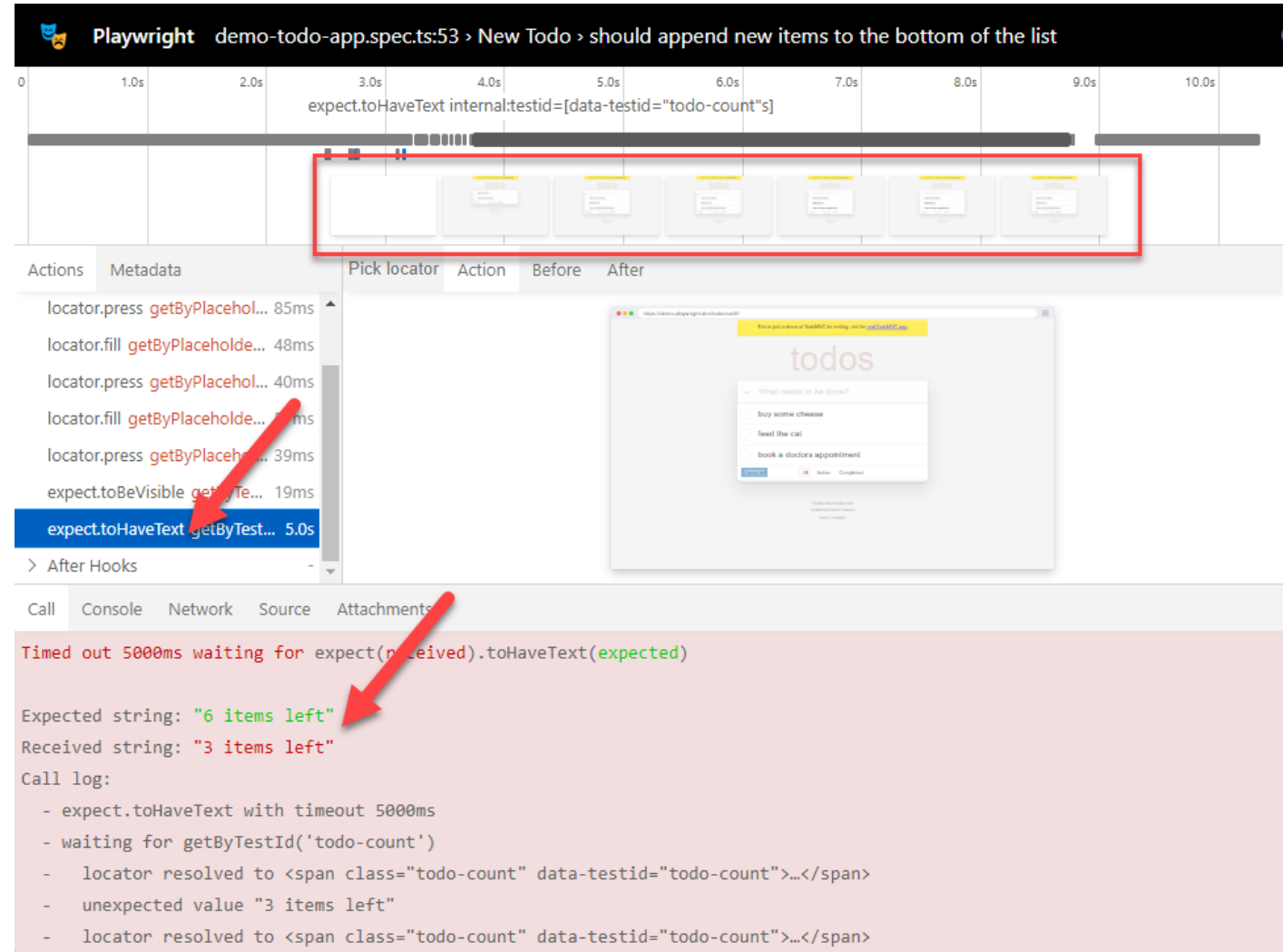
- Use Plugin

Hosted Trace File Viewer

- <https://trace.playwright.dev>



Playwright Trace Viewer is a Progressive Web App, it does not send your trace anywhere, it opens it locally.



Report List

Playwright Test comes with a few built-in reporter.

HTML Report

- `npx playwright test --reporter=html`

List Report

- `npx playwright test --reporter=list`

Line Report

- `npx playwright test --reporter=line`

Dot Report

- `npx playwright test --reporter=dot`

1

Q

All 24 Passed 23 **Failed 1** Flaky 0 Skipped 0

Project: chromium Total time: 12.0s

▼ demo-todo-app.spec.ts

- ✗ New Todo > should append new items to the bottom of the list 7.2s
demo-todo-app.spec.ts:53
- ✓ New Todo > should allow me to add todo items 1.3s
demo-todo-app.spec.ts:14
- ✓ New Todo > should clear text input field when an item is added 1.4s
demo-todo-app.spec.ts:40

2

```
Running 6 tests using 6 workers

✓ 1 [chromium] > demo-todo-app.spec.ts:14:7 > New Todo >
✓ 2 [chromium] > demo-todo-app.spec.ts:53:7 > New Todo >
✓ 3 [firefox] > demo-todo-app.spec.ts:14:7 > New Todo >
✓ 4 [firefox] > demo-todo-app.spec.ts:40:7 > New Todo >
✓ 5 [chromium] > demo-todo-app.spec.ts:40:7 > New Todo >
✓ 6 [firefox] > demo-todo-app.spec.ts:53:7 > New Todo >

6 passed (11.1s)
```

3

```
Running 6 tests using 6 workers
.....
6 passed (11.1s)
```


Playwright UI Mode

Explore, run and debug tests with a time travel experience complete with watch mode

CLI

```
npx playwright test testname.ts --ui
```

DOM Inspection

- page structure, inspect elements, view attributes.

Console Logs

- JavaScript-generated logs, errors, and warnings.

Network Monitoring

- Requests, timings, inspect headers, payloads.

Interactivity

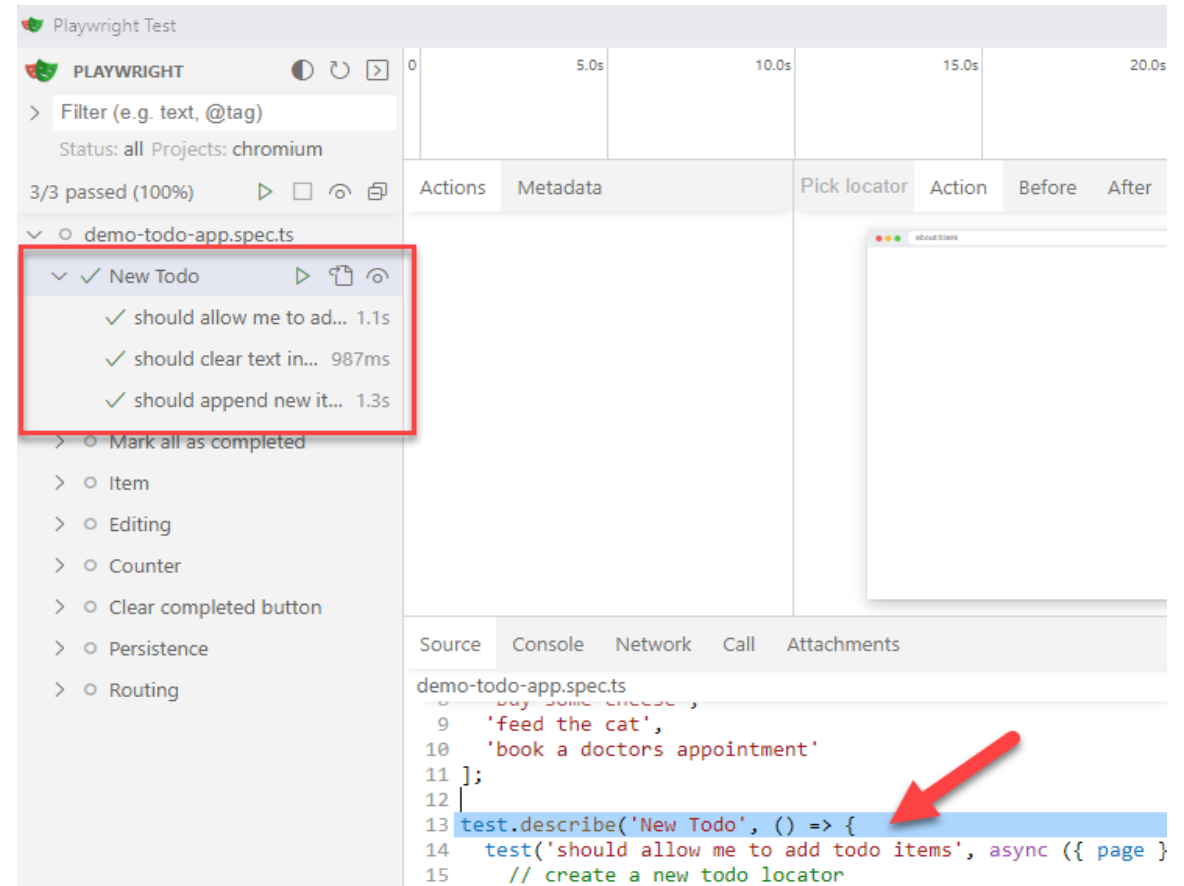
- Trigger events, clicks, real-time page responses.

Element Highlighting

- Hover over elements to highlight them.

Source Code Access

- View HTML, CSS, JS source for debugging.



Why use locators?

Locators represents a view to the element(s) on the page. It captures the logic sufficient to retrieve the element at any given moment.

<https://playwright.dev/docs/api/class-locator>

How do I use locators?

Instead of this

```
await page.click('text="Login"');
```

Use this

```
await page.locator('text="Login"').click();
```

Operations on the target DOM element will fail

```
// Throws if there are several buttons in DOM  
await page.locator('button').click();
```

This results in 3 outcomes:

- Test works as expected
- Selector does not match anything and test fails
- Multiple elements match the selector, test fails with helpful error message